DysWrite: A Mobile Application for Early Detection of Dyslexia

Code Quality & Best Practices Audit — Software Engineering Principles Compliance Report

Course	4th-Year Computer Science / Software Engineering
Institution	DMMMSU – South La Union Campus, College of Computer Science
Thesis Project	DysWrite: Mobile Dyslexia Pre-Screening Application
Repository	dyswrite-prescreen (group GitHub repository, root/main folder)
Research Group	Gerrald A. Bicera • Angeline G. Garcia • John Albert C. Lachica • Regine J. Velasquez • Rachel
Adviser	Prof. Belinda D. Celestial
Reference Frameworks	Coding Standards • 15 Principles of Quality Software • Defensive Programming • Error/Exception Handling

This report documents which parts of the dyswrite-prescreen repository implement — and which parts fail to fully implement — the four principle sets covered in Module 2 of the course. Each finding is traced to a specific source file and function, quoted in a short excerpt, and explained against the source lecture notes.

1. Scope and Methodology

The audited artifact is the **dyswrite-prescreen** repository: 8 source modules under *src/* (config, exceptions, utils, dataset, model, gradcam, train, infer), 2 unit-test modules under *tests/*, and a README with setup/run instructions. This is DysWrite's own project code, uploaded to the group's GitHub repository per the assignment's requirement that other group members be able to clone and run it on their own devices.

Every file was read in full and checked against four reference frameworks covered in class:

- **Professional Coding Standards & Best Practices** (naming, readability, DRY, comments/documentation, headers)
- **15 Principles for Building Quality Software** (architecture, edge cases, testing, code hygiene, rollback-ability)
- **Defensive Programming** (eliminating assumptions, validating input, consistent guard logic)
- **Error / Exception Handling** (try/catch/finally usage, exception categories, graceful recovery)

For each framework, findings are split into **(A) Implemented** — principles the code demonstrably follows, with file/function-level evidence — and **(B) Not Implemented / Gaps** — principles the code violates, omits, or only partially satisfies. A consolidated compliance matrix and recommendations close the report.

Legend	**IMPLEMENTED** – clearly present and correctly applied
	PARTIAL – present but inconsistent or incomplete
	NOT IMPLEMENTED – absent / violated

2. Professional Coding Standards & Best Practices

2.A — Parts of the Code That Implement This Principle

Standardized Module Headers on Every File	**IMPLEMENTED**	All 10 files under src/ and tests/

```
"""
Module Name : dataset.py
Project : DysWrite Pre-Screening Engine
Created : 2026-08-17
Author : DysWrite Research Group (DMMMSU-SLUC College of CS)
Summary : Loads handwriting sample images from disk into a
PyTorch Dataset. Applies defensive checks...
Functions : build_transform()
Classes : HandwritingDataset(Dataset)
"""
```

Every module opens with a header stating the module name, project, creation date, author, a summary, and the functions/classes it defines — a direct implementation of the Coding Standards lecture's Part 2, Item 11 (“Standardize Headers for Different Modules”), which was completely absent from the group's earlier reference notebook.

Centralized Constants — No Magic Numbers	**IMPLEMENTED**	src/config.py, used by dataset.py, model.py, train.py, gradcam.py

```
BATCH_SIZE = 16
LEARNING_RATE = 1e-4
NUM_EPOCHS = 15
TRAIN_SPLIT = 0.8
VAL_SPLIT = 0.1
EARLY_STOPPING_PATIENCE = 3
GRADCAM_TARGET_LAYER = "features.12"
MIN_CONFIDENCE_THRESHOLD = 0.55
```

Every hyperparameter and hardcoded string that was previously scattered inline (split ratios, batch size, the Grad-CAM target-layer name) now lives in one file. Any other module that needs one of these values imports it from *config* rather than re-typing it — directly resolving the “magic numbers” gap flagged in the earlier notebook audit and matching the readability guidance in Coding Standards Part 2, Item 2.

Docstrings on Every Public Function and Class	**IMPLEMENTED**	src/utils.py — safe_open_image()

```
def safe_open_image(path):
"""
Defensively open an image file and convert it to RGB.
...
Args:
path: filesystem path to the image file.
Returns:
A PIL.Image.Image object in RGB mode.
Raises:
InvalidImageError: if the file does not exist or cannot
be decoded as an image.
"""
```

Every public function across the eight source modules documents its parameters, return value, and the exceptions it can raise, resolving the “missing docstrings” gap identified in the earlier notebook audit and matching the “Prioritize Documentation” guidance in Coding Standards Part 2, Item 5.

DRY — One Shared Epoch Function Instead of Duplicated Loops	**IMPLEMENTED**	src/train.py — _run_one_epoch()

```
def _run_one_epoch(model, loader, criterion, optimizer, device, train: bool):
"""
Shared epoch loop for both training and validation (DRY: avoids
two near-identical copies of this loop, unlike the reference
notebook where the corrupted-label guard existed only in the
training branch).
"""
model.train() if train else model.eval()
...
```

Training and validation share a single *_run_one_epoch()* function toggled by a boolean flag, instead of two separately maintained copies of the same loop. This directly resolves the earlier finding that the validation loop was missing a guard the training loop had — with one shared function, both paths get every guard automatically.

2.B — Parts of the Code That Do NOT Implement This Principle

Dead Constant — LOG_DIR Is Defined but Never Used	**NOT IMPLEMENTED**	src/config.py, line 26

```
LOG_DIR = PROJECT_ROOT / "logs"
```

LOG_DIR is declared alongside *CHECKPOINT_DIR* and *OUTPUT_DIR*, but unlike those two, no file in *src/* ever references it — *utils.get_logger()* only logs to stdout via *StreamHandler*, never to a file. This is the same class of issue flagged in the earlier notebook audit (Coding Standards Part 2, Item 8: unused/temporary code left in place) recurring in the new project, just in a smaller form.

Silent Skipping of Non-Image Files With No Logged Warning	**NOT IMPLEMENTED**	src/dataset.py — _index_samples()

```
valid_extensions = {".png", ".jpg", ".jpeg", ".bmp"}
for cls in self.classes:
class_dir = self.root_folder / cls
for image_path in class_dir.iterdir():
if image_path.suffix.lower() in valid_extensions:
samples.append((image_path, self.class_to_idx[cls]))
# non-matching files are skipped with no log line
```

Files with an unrecognized extension (e.g. a stray *.txt* or *.gif* dropped into a class folder) are silently excluded from the dataset with no warning printed. A developer troubleshooting ‘why is my dataset smaller than expected’ has no clue from the output alone — a smaller-scale version of the same readability/observability gap the Coding Standards notes' comment guidance (Part 2, Item 5) is meant to prevent.

3. 15 Principles for Building Quality Software

3.A — Parts of the Code That Implement This Principle

Principle 5 — Edge Cases Handled Consistently in Both Loops	**IMPLEMENTED**	src/train.py — _run_one_epoch(), used for both train and val

```
for inputs, labels in loader:
valid_mask = labels != -1
if valid_mask.sum() == 0:
continue # entire batch was corrupted/unreadable; skip safely
inputs, labels = inputs[valid_mask].to(device), labels[valid_mask].to(device)
```

Because both training and validation now call the same *_run_one_epoch()* function, the corrupted-image guard automatically applies to both — closing the specific inconsistency the earlier audit found between the notebook's training and validation loops. This is Principle 5 (“think through edge cases upfront”) applied consistently rather than in only one code path.

Principle 6 — Do Not Assume; Validate Configuration Early	**IMPLEMENTED**	src/config.py — validate_split_ratios(), called at import time

```
def validate_split_ratios() -> None:
test_split = 1.0 - TRAIN_SPLIT - VAL_SPLIT
if test_split <= 0:
raise ValueError(
f"Invalid split configuration: TRAIN_SPLIT ({TRAIN_SPLIT}) + "
f"VAL_SPLIT ({VAL_SPLIT}) leaves no room for a test split..."
)

validate_split_ratios() # runs the moment config.py is imported
```

Rather than assuming the split ratios will always be sensible, this check runs the instant the config module is imported — anywhere in the project — and fails loudly with a clear message if the numbers don't add up. This matches Principle 6's instruction to clarify uncertainty and avoid silent assumptions.

Principle 12 — Real, Automated Unit Tests	**IMPLEMENTED**	tests/test_dataset.py, tests/test_model.py (9 test cases total)

```
def test_class_mismatch_raises_class_mismatch_error(self):
root = self.tmp_dir / "wrong_classes"
(root / "cat").mkdir(parents=True)
(root / "dog").mkdir(parents=True)
with self.assertRaises(ClassMismatchError):
HandwritingDataset(str(root))

def test_corrupted_image_yields_sentinel_label(self):
...
self.assertTrue(found_sentinel, "Corrupted image should yield -1")
```

Unlike the earlier notebook, which had zero automated tests, this project has real unit tests that construct temporary datasets, deliberately corrupt a file, and assert the defensive code actually behaves as documented — a direct implementation of Principle 12 (“Write Unit Tests… if your code is not testable, it needs to be refactored so that it is”).

Principle 10 — Early-Stopping Logic Fully Wired, Not Left as Scaffolding	**IMPLEMENTED**	src/train.py — run_training()

```
if val_loss < best_val_loss - config.EARLY_STOPPING_MIN_DELTA:
best_val_loss = val_loss
epochs_without_improvement = 0
save_checkpoint(model, config.CHECKPOINT_DIR / "best_model.pth")
else:
epochs_without_improvement += 1
if epochs_without_improvement >= config.EARLY_STOPPING_PATIENCE:
logger.info("Early stopping triggered at epoch %d.", epoch)
break
```

The earlier notebook audit flagged early-stopping variables that were declared but never actually used. Here, the counter increments, compares against the configured patience, and actually breaks the loop — the feature was finished rather than left as unused scaffolding, resolving that specific Principle 10 violation.

3.B — Parts of the Code That Do NOT Implement This Principle

Principle 12 — Test Coverage Is Incomplete	**PARTIAL**	tests/ directory covers only dataset.py and model.py

gradcam.py, *train.py*, *infer.py*, and *utils.py* have no dedicated unit tests at all — only *HandwritingDataset* and the model build/checkpoint functions are covered. The early-stopping logic praised above, for example, is exercised only by manually running training, not by an automated test that could catch a regression on every commit, which is what Principle 12 actually asks for.

Principle 3 — Still Organized by Layer, Not by Feature	**NOT IMPLEMENTED**	src/ directory structure

```
src/
config.py
exceptions.py
utils.py
dataset.py
model.py
gradcam.py
train.py
infer.py
```

Files are grouped by what kind of thing they are (one file per architectural concern) rather than by feature. For a project this size that is a defensible, common choice, but it is still the layer-based organization Principle 3 contrasts against — “package by feature” would instead group, for example, all pre-screening-related code (model + Grad-CAM + inference) into one feature module separate from a future feature (e.g. a progress-tracking dashboard).

Principle 15 — No Versioning on Saved Checkpoints	**NOT IMPLEMENTED**	src/train.py — run_training()

```
save_checkpoint(model, config.CHECKPOINT_DIR / "best_model.pth")
```

Every improved checkpoint overwrites the same *best_model.pth* file. If a new training run produces a worse model due to a bug, there is no previous version to roll back to — the old “best” model is already gone. Principle 15 (“make rolling back easy”) calls for tagged or timestamped checkpoints; none exist here.

Principle 14 — No Persistent Log for Post-Crash Review	**NOT IMPLEMENTED**	src/utils.py — get_logger()

```
handler = logging.StreamHandler(sys.stdout)
# no logging.FileHandler(config.LOG_DIR / ...) is ever added
```

All log output goes only to the console. If *train.py* crashes overnight during an unattended run, there is no saved log file to review afterward — only whatever was visible in the terminal at the time, which is easily lost. Principle 14 (“monitor crash reports”) assumes some persistent record exists to review; here, none does, despite *config.LOG_DIR* already being defined for this exact purpose.

4. Defensive Programming

The reference lecture is PHP-specific, but its two pillars — eliminating assumptions about incoming data, and consistently validating that data — apply directly to this project's handling of external image files and configuration.

4.A — Parts of the Code That Implement This Principle

num_classes Cross-Checked Against the Dataset Folder on Disk	**IMPLEMENTED**	src/dataset.py — HandwritingDataset.__init__()

```
discovered = sorted(p.name for p in self.root_folder.iterdir() if p.is_dir())
if set(discovered) != set(config.CLASS_NAMES):
raise ClassMismatchError(
f"Class folders discovered on disk {discovered} do not match "
f"config.CLASS_NAMES {config.CLASS_NAMES}. Update config.py or "
f"fix the dataset folder structure before continuing."
)
```

The earlier notebook audit flagged that *num_classes* was hardcoded and never checked against the dataset's actual folders. Here, that exact assumption is eliminated: the dataset constructor discovers the real folders and raises a specific, named exception the moment they disagree with configuration, instead of silently training against a mismatched label space.

Single Point of Truth for “Is This a Valid Image?”	**IMPLEMENTED**	src/utils.py — safe_open_image()

```
if not path.exists():
raise InvalidImageError(f"Image file does not exist: {path}")
try:
with Image.open(path) as img:
return img.convert("RGB")
except (OSError, UnidentifiedImageError) as exc:
raise InvalidImageError(f"Could not decode image at {path}: {exc}") from exc
```

Both *dataset.py* (bulk loading) and *infer.py* (single-image CLI input) call this same function, so the same defensive validation applies everywhere an image enters the system — no code path assumes a file is a valid image without checking, matching the PHP notes' “eliminate assumptions” pillar applied consistently rather than in just one place.

Guarding Against Division by Zero on Degenerate Batches/Epochs	**IMPLEMENTED**	src/train.py — _run_one_epoch()

```
if total == 0:
logger.warning("An entire %s split produced zero valid samples this epoch.",
"training" if train else "validation")
return float("inf"), 0.0

avg_loss = running_loss / total
accuracy = correct / total
```

This directly resolves the latent *ZeroDivisionError* risk the earlier notebook audit flagged: if every sample in a split turns out to be corrupted, the code now returns a defined neutral result and logs a warning instead of crashing with an unhandled arithmetic error.

4.B — Parts of the Code That Do NOT Implement This Principle

Grad-CAM Assumes a Correctly-Shaped Single-Image Tensor	**NOT IMPLEMENTED**	src/gradcam.py — generate_gradcam_overlay()

```
def generate_gradcam_overlay(model, image_tensor, original_image,
target_layer=config.GRADCAM_TARGET_LAYER):
...
outputs = model(image_tensor) # assumes image_tensor is shape (1, 3, H, W)
predicted_class = int(outputs.argmax(dim=1)[0].item())
```

The function never checks that *image_tensor* actually has a batch dimension of 1 (a single image) before indexing *outputs.argmax(dim=1)[0]*. If a caller ever passes a full batch by mistake, this silently returns only the first image's result instead of raising a clear error — an unvalidated assumption about the caller's input shape.

infer.py Assumes the Output Path Is Always Writable	**NOT IMPLEMENTED**	src/infer.py — main()

```
plt.tight_layout()
plt.savefig(args.output, dpi=150) # no validation the directory is writable
plt.close(fig)
```

If *args.output* points to a directory that doesn't exist or the process lacks write permission, *plt.savefig()* raises an unhandled error — after the model has already run and produced a valid prediction. The result is silently discarded instead of being reported to the user in a friendly way, an assumption that the output path is always valid.

5. Error / Exception Handling

5.A — Parts of the Code That Implement This Principle

A Real, Well-Defined Exception Hierarchy	**IMPLEMENTED**	src/exceptions.py

```
class DysWriteError(Exception):
"""Base class for all custom exceptions raised by this project."""

class DatasetError(DysWriteError): ...
class ClassMismatchError(DysWriteError): ...
class ModelCheckpointError(DysWriteError): ...
class InvalidImageError(DysWriteError): ...
```

Instead of letting every failure surface as a generic, unspecific built-in exception, the project defines its own hierarchy rooted at *DysWriteError*. This lets calling code catch precisely the failure it knows how to handle (e.g. *ModelCheckpointError* specifically) rather than a bare *except Exception*, matching the “well-defined hierarchy” guidance in the Defensive Programming notes (Section II) and the checked/unchecked exception taxonomy in the Exception Handling notes (Section V.1).

Checkpoint Loading Is Fully Guarded (Fixes the Earlier Notebook's Gap)	**IMPLEMENTED**	src/model.py — load_checkpoint()

```
if not path.exists():
raise ModelCheckpointError(f"Checkpoint file not found: {path}")
try:
state_dict = torch.load(path, map_location=device)
model.load_state_dict(state_dict)
except (RuntimeError, EOFError, OSError) as exc:
raise ModelCheckpointError(f"Failed to load checkpoint {path}: {exc}") from exc
```

The earlier notebook audit's single biggest Error-Handling finding was that *torch.load()* was called with no guard at all. Here, existence is checked first, and the load itself is wrapped in try/except covering the specific exception types PyTorch can raise for a missing or corrupted file, re-raised as one clear, project-specific error.

finally Block Guarantees Cleanup/Reporting Regardless of Outcome	**IMPLEMENTED**	src/train.py — run_training()

```
start_time = time.time()
try:
for epoch in range(1, config.NUM_EPOCHS + 1):
...
finally:
duration_min = (time.time() - start_time) / 60
logger.info("Training run finished after %.2f minutes.", duration_min)
```

This is a direct application of the *finally* pattern demonstrated in the Exception Handling notes (Section IV, Example 2): the duration is reported whether training completes normally, is early-stopped, or crashes partway through — something the earlier notebook never did anywhere.

CLI Entry Point Catches Multiple Specific Exception Types	**IMPLEMENTED**	src/infer.py — main()

```
except ModelCheckpointError as exc:
logger.error("Could not load model: %s", exc)
return 1
...
except InvalidImageError as exc:
logger.error("Could not read input image: %s", exc)
return 1
...
except DysWriteError as exc:
logger.error("Pre-screening failed: %s", exc)
return 1
except RuntimeError as exc:
logger.error("An unexpected model error occurred: %s", exc)
return 1
```

Each stage of the pipeline (model loading, image reading, prediction) is wrapped in its own try/except catching the specific exception that stage can raise, with a distinct, actionable log message and a clean exit code — rather than one broad catch-all or, as in the reference notebook, no handling at all.

5.B — Parts of the Code That Do NOT Implement This Principle

Grad-CAM Hooks Are Not Guaranteed to Be Removed on Failure	**NOT IMPLEMENTED**	src/gradcam.py — generate_gradcam_overlay()

```
cam_extractor = GradCAM(model, target_layer=target_layer)
outputs = model(image_tensor)
...
activation_map = cam_extractor(predicted_class, outputs) # if this raises...
...
cam_extractor.remove_hooks() # ...this line never runs
```

If anything between creating *cam_extractor* and calling *cam_extractor.remove_hooks()* raises an exception, the forward hooks registered on the model are never removed — they stay attached, silently affecting every future forward pass. This is exactly the kind of guaranteed-cleanup scenario a *finally* block exists for (Exception Handling notes, Section IV), and it is missing here despite being correctly used elsewhere in the same project (*train.py*, Section 5.A).

Unguarded File Save in the CLI's Final Step	**NOT IMPLEMENTED**	src/infer.py — main()

```
plt.savefig(args.output, dpi=150)
plt.close(fig)
logger.info("Saved annotated result to %s", args.output)
```

Unlike every earlier stage of *main()* (model loading, image reading, prediction), the final save step has no try/except. A bad output path throws an unhandled exception after a correct prediction was already computed — the one stage in this otherwise well-guarded function that was left unprotected.

Exception Paths in train.py and infer.py Are Untested	**NOT IMPLEMENTED**	tests/ directory

The custom exceptions in *exceptions.py* are only exercised indirectly through *test_dataset.py* and *test_model.py* (e.g. *ClassMismatchError*, *ModelCheckpointError*). There is no test confirming that *infer.py*'s *main()* actually returns exit code 1 and logs the right message when, say, a checkpoint is missing — the CLI's own error-handling behavior is unverified by automation, only by the code's own logic being read manually.

6. Consolidated Compliance Matrix

Quick-reference summary of every finding above, ordered by framework.

Framework	Finding	File	Status
Coding Standards	Standardized module headers on every file	all files	**Implemented**
Coding Standards	Centralized constants (no magic numbers)	config.py	**Implemented**
Coding Standards	Docstrings on every public function/class	all files	**Implemented**
Coding Standards	DRY via shared _run_one_epoch()	train.py	**Implemented**
Coding Standards	Dead LOG_DIR constant, never used	config.py	**Not Implemented**
Coding Standards	Non-image files skipped with no log warning	dataset.py	**Not Implemented**
Quality Software (15 Principles)	Edge-case guard applied consistently (train+val)	train.py	**Implemented**
Quality Software (15 Principles)	Config validated at import time	config.py	**Implemented**
Quality Software (15 Principles)	Real automated unit tests	tests/	**Implemented**
Quality Software (15 Principles)	Early stopping fully wired, not scaffolding	train.py	**Implemented**
Quality Software (15 Principles)	Test coverage incomplete (4 of 8 modules untested)	tests/	**Partial**
Quality Software (15 Principles)	Still organized by layer, not by feature	src/	**Not Implemented**
Quality Software (15 Principles)	No checkpoint versioning / rollback path	train.py	**Not Implemented**
Quality Software (15 Principles)	No persistent log file for crash review	utils.py	**Not Implemented**
Defensive Programming	num_classes cross-checked against dataset folders	dataset.py	**Implemented**
Defensive Programming	Single shared image-validation function	utils.py	**Implemented**
Defensive Programming	Division-by-zero guarded on empty splits	train.py	**Implemented**
Defensive Programming	Grad-CAM assumes correct tensor shape, unchecked	gradcam.py	**Not Implemented**
Defensive Programming	Output path writability never validated	infer.py	**Not Implemented**
Error/Exception Handling	Well-defined custom exception hierarchy	exceptions.py	**Implemented**
Error/Exception Handling	Checkpoint loading fully guarded	model.py	**Implemented**
Error/Exception Handling	finally block guarantees duration reporting	train.py	**Implemented**
Error/Exception Handling	CLI catches multiple specific exception types	infer.py	**Implemented**
Error/Exception Handling	Grad-CAM hooks not cleaned up on failure	gradcam.py	**Not Implemented**
Error/Exception Handling	Final image save step is unguarded	infer.py	**Not Implemented**
Error/Exception Handling	CLI error paths untested by automation	tests/	**Not Implemented**

7. Recommendations for Next Iteration

The following fixes are recommended for the group's next commit to the dyswrite-prescreen repository, in priority order:

- **Wrap the Grad-CAM extraction call in try/finally** so *cam_extractor.remove_hooks()* always runs, even if activation-map extraction fails partway through (Section 5.B).
- **Guard the final plt.savefig() call in infer.py** with its own try/except so a bad output path reports a clear error instead of discarding an already-successful prediction (Section 5.B).
- **Extend unit tests to gradcam.py, train.py, and infer.py**, at minimum testing that each raises the correct custom exception on its known failure paths (Sections 3.B, 5.B).
- **Add a timestamp or fold/version number to saved checkpoints** (e.g. best_model_2026-08-17_v1.pth) so a bad training run doesn't silently overwrite the last good model (Section 3.B).
- **Wire config.LOG_DIR into utils.get_logger()** via a logging.FileHandler so training runs leave a reviewable log file, and remove it from config.py if the team decides not to use it (Sections 2.B, 3.B).
- **Log a warning in dataset.py when a file is skipped** for having an unrecognized extension, so dataset-size discrepancies are easy to trace (Section 2.B).

8. Conclusion

The dyswrite-prescreen repository resolves every major gap identified in the group's earlier audit of the third-party reference notebook: checkpoint loading is now guarded, the validation loop shares the same corrupted-data guard as training, class counts are cross-checked against the dataset on disk, and real unit tests exist where none did before. At the same time, this rewrite introduces its own smaller, more localized gaps — most notably that Grad-CAM's cleanup step (*remove_hooks()*) is not protected by the same *finally* pattern the project correctly uses in *train.py*, and that test coverage, while real, does not yet extend to every module. These are meaningfully smaller-scope issues than the ones found previously, and the recommendations in Section 7 close them directly.

Repository audited: dyswrite-prescreen (DysWrite group's own GitHub repository, root/main folder). Model architecture approach informed by Robaa et al., cited in DysWrite Chapter 2 Literature Cited.